

solution

1.奶龙的影分身

题意:

输入并输出 26 个数字的和即可

解法:

循环模拟即可

notice:

注意数据 $26 \times 10^8 > 2147483647$, 超出了 *int* 范围, 答案变量应使用 *longlong*

std: (by Today)

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    std::ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    long long sum=0;
    for(int i=0;i<26;i++){
        int x;
        cin>>x;
        sum+=x;
    }
    cout<<sum;
    return 0;
}
```

时间复杂度: $O(1)$

2.奶龙的疑惑

题意:

计算从 l 到 r 每个数的 k 次方的和, 把和对 2 取余就是答案

解法:

即为判断最后和的奇偶性 很容易注意到, $mod2$ 是统计二进制位数的最低位, 且不会因为计算过程中的进位产生变化 另外可以发现, 每个数的 k 次方($k \geq 1$) 的时候, 数字的奇偶性一定不会改变, 所以相当于统计 l 到 r 中奇数的数量, 对 2 取余就是最终的答案 而对于 $k = 0$ 时, 无论谁的 k 次方都是 1, 对这种情况特判即可

notice:

数字会超过 *int* 范围

std: (by Today)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
void solve(){
    int l,r,k;
    cin>>l>>r>>k;
    if(k==0){
        cout<<(r-l+1)%2<<"\n";
        return;
    }
    cout<<((l+r)*(r-l+1)/2)%2<<"\n";
}
signed main() {
    std::ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    cin>>t;
    while(t--){
        solve();
    }
}
```

时间复杂度: $O(1)$

3. 奶龙的难题

题意:

本质上相当于算出所有数对 (a, b) 的 $\&$ 运算之和

解法:

可以观察到, 如果暴力枚举所有数对然后计算的时间复杂度是 $O(n^2)$, 会超时

可以通过贡献法优化 很容易观察到, 把数字拆成二进制后每一位上的运算都是独立的 由于 $\&$ 运算的性质, 只有数对 (a, b) 在第 j 位都是 1 的时候才会对答案的二进制第 j 位产生贡献, 贡献为这个 n 个数中, 第 j 位为 1 的数字相互两两配对的方案数, 即

$$cnt_j * (cnt_j - 1) / 2$$

notice:

注意答案可能爆 *int*

std: (by Today)

```

#include<bits/stdc++.h>
using namespace std;
#define int long long
void solve(){
    int n;
    cin>>n;
    vector<int>a(n);
    for(int &x:a)cin>>x;
    int ans=0;
    for(int i=0;i<32;i++){
        int sum=0;
        for(int j=0;j<n;j++){
            if((a[j]>>i)&1)sum++;
        }
        ans+=(sum*(sum-1)/2)*(1LL<<i);
    }
    cout<<ans<<"\n";
}
signed main() {
    std::ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    cin>>t;
    while(t--){
        solve();
    }
    return 0;
}

```

时间复杂度: $O(n \times \log MAXN)$

4. 奶龙的数字交换

题意:

通过两种操作把 s 变为 t

解法:

可以把可达的数抽象成节点, 操作抽象成两种边, 然后使用 bfs 搜索即可

std: (loopy)

```

#include <bits/stdc++.h>
using namespace std;
using ll=long long;
ll c[2000009],vis[3000009];

```

```

int main(){
    ll s,t,a,b;cin>>s>>t>>a>>b;
    queue<ll>q;
    q.push(s);
    vis[s]=1;
    ll ans=-1;
    while(!q.empty())
    {
        ll x=q.front();
        q.pop();
        if(x==t){ans=c[x];break;}
        ll x1=x%a;
        ll x2=x+b;
        if(vis[x1]==0){
            c[x1]=c[x]+1;
            q.push(x1);
            vis[x1]=1;
        }
        if(vis[x2]==0&&x2<=2e6){
            c[x2]=c[x]+1;
            q.push(x2);
            vis[x2]=1;
        }
    }
    cout<<ans;
}

```

时间复杂度: $O(2 \times 10^6)$

5. 奶龙的 🍔

题意:

题意：给定若干种口味的汉堡，两人轮流取汉堡，奶龙先手。奶龙最多可用 k 次技能 ($k \leq 10$)，每次技能可一次性取走场上某一整类口味汉堡，普通操作只能单取一个。计分规则：拿到过某口味得 1 分，包揽某口味全部汉堡再额外加 1 分。双方均采用最优策略：奶龙最大化得分，对手最小化其得分，求奶龙最终能拿到的最大分数。

解法:

用桶统计每种口味汉堡的数量，将口味分为三类：

- **sum1**：数量为1的口味数（独一份汉堡）。
- **sum2**：数量为2的口味数。
- **sum3**：数量 ≤ 3 的口味数。

基础得分：

- $sum1$ 类汉堡优先级最高，奶龙与微风会优先抢夺。奶龙获得 $\lceil \frac{sum1}{2} \rceil$ 个，每份得 2 分，贡献为 $\lceil \frac{sum1}{2} \rceil \times 2$ 。
- 对于 $sum2 + sum3$ 类汉堡，微风用对称策略阻止奶龙独占，贡献为 $sum2 + sum3$ 。

技能（恶龙咆哮）得分：

- 若 **sum1 为偶数**（奶龙先手）：
 1. 优先对 sum_2 使用技能，只能使用一次可获2分，如果 $sum_2 \geq 1$ 并且 $k > 0$, 贡献为 1 并且 $k -= 1$, 否则为 0。
 2. 剩余技能用于 sum_3 ，每2个口味用1次技能最优，贡献为 $\lfloor \min(\frac{sum_3}{2}, rem_k) \rfloor$ 。
- 若 **sum1 为奇数**（奶龙后手）：
 - 技能只能用于 sum_3 ，每2个口味用1次技能最优，贡献为 $\lfloor \min(\frac{sum_3}{2}, k) \rfloor$ 。

总得分 = 基础分 + 技能分。

std:(by today)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    std::ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin>>t;
    while(t--){
        int n,k;
        cin>>n>>k;
        vector<int>tong(n+1);
        for(int i=0,x;i<n;i++){
            cin>>x;
            tong[x]++;
        }
        int sum1=0,sum2=0,sum3=0;
        for(int i=1;i<=n;i++){
            if(tong[i]==1)sum1++;
            if(tong[i]==2)sum2++;
            if(tong[i]>2)sum3++;
        }
        //奶龙微风抢一。
        int ans=0;
        ans+=(sum1+1)/2;
        ans*=2;
        ans+=(sum2+sum3);
        if(sum1%2==0){
            //奶龙先手，优先对sum2进行一次恶龙咆哮
            if(sum2>0){
                ans++,k--;
                ans+=min(sum3/2,k);
            }else{
                ans+=min((sum3+1)/2,k);
            }
        }else{
            //微风先手
            ans+=min(sum3/2,k);
        }
        cout<<ans<<"\n";
    }
}
```

```
    }
}
```

时间复杂度: $O(n)$

6. GoryK 的数列划分

题意:

只用给定数组生成 k 个子数组, 让他们的极差和最小

解法:

可以做一步问题转化, 计算一个数组的极差不仅可以用 $max - min$ 也可以把数组排序后, 相邻数的差值之和也是极差 我们把原始数组排序, 把每个数抽象成一个节点, 边权为相邻数之差, 现在的数组可以看作一个特殊形态的树, 想要完成 k 次划分我们只需要删除 $k - 1$ 条边, 贪心的想, 删掉最大的 $k - 1$ 条边即可 于是我们把 $n - 1$ 条边排序后计算即可 本题的思想源于最小生成树, 但是 std 写到一半发现一个简单贪心即可秒掉 OvO

std: (by GoryK)

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int long long

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#endif

void solve() {
    int n, k;
    cin >> n >> k;
    vector<int> vec(n + 1);
    for (int i = 1; i <= n; i ++){
        cin >> vec[i];
    }

    sort(vec.begin() + 1, vec.end());

    vector<int> val;
    for (int i = 2; i <= n; i ++){
        val.push_back(vec[i] - vec[i - 1]);
    }

    sort(val.begin(), val.end());

    int ans = 0;
    for (int i = 0; i < n - k; i ++){
        ans += val[i];
    }
}
```

```

    }

    cout << ans << "\n";
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    // std::cin >> tt;
    while (tt-- && ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}

```

时间复杂度: $O(n)$

7. GoryK 的城市光缆

题意:

给定 n 个二维节点, 定义连接两个节点的代价为其间欧氏距离的平方, 求生成 k 个连通块的最小代价

解法:

经典的最小生成树题目, 由于要生成 k 个连通块, 我们在跑一边最小生成树的基础上删掉 $k - 1$ 条边即可, 由于求最小, 我们贪心的减掉 $k - 1$ 条最大的边即可

std:(by GoryK)

```

// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int long long

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#endif

// DSU
class DSU {
public:
    std::vector<int> f, siz;
public:
    DSU() {}
}

```

```
DSU(int n) {
    init(n);
}

void init(int n) {
    f.resize(n);
    std::iota(f.begin(), f.end(), 0);
    siz.assign(n, 1);
}

int find(int x) {
    while (x != f[x]) {
        x = f[x] = f[f[x]];
    }
    return x;
}

bool same(int x, int y) {
    return find(x) == find(y);
}

bool merge(int x, int y) {
    x = find(x);
    y = find(y);
    if (x == y) {
        return false;
    }
    siz[x] += siz[y];
    f[y] = x;
    return true;
}

int size(int x) {
    return siz[find(x)];
}
};

void solve() {
    int n, k;
    cin >> n >> k;
    vector<array<int, 2>> vec(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> vec[i][0] >> vec[i][1];
    }

    auto calc = [&](int x1, int y1, int x2, int y2) -> int {
        int dx = abs(x2 - x1), dy = abs(y2 - y1);
        return dx * dx + dy * dy;
    };

    vector<array<int, 3>> edges;
    for (int i = 1; i < n; i++) {
        for (int j = i + 1; j <= n; j++) {
            int w = calc(vec[i][0], vec[i][1], vec[j][0], vec[j][1]);
```



```
        edges.push_back({i, j, w});
    }
}

sort(edges.begin(), edges.end(), [&](const auto &a, const auto &b) {
    return a[2] < b[2];
});

DSU dsu(n + 1);
int cnt = 0, lim = (n - 1) - (k - 1), ans = 0;
for (const auto &[u, v, w] : edges) {
    if (dsu.same(u, v)) continue;
    ans += w;
    dsu.merge(u, v);
    if (++cnt == lim) break;
}

cout << ans << "\n";
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    // std::cin >> tt;
    while (tt-- && ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}
```

时间复杂度: $O(n^2 \log n)$